

something called a Laravel configuration file which is stored at `config/app.php`. On opening this file, you'll be shown a list of laravel service providers. Add a provider from this list.

Step 4 – After adding your providers, you will need to add aliases in the same file for HTML and Form. What do we mean by an alias? As the name suggests, they are alternative icons that you can assign for the same file in different folders. It is a great feature to be able to add aliases as some files fall into multiple categories that each of the folders contain.

The following codes can be used to add an alias for form:

```
'Form' => Illuminate\Html\FormFacade::class,
```

To add an alias for html

```
'Html' => Illuminate\Html\HtmlFacade::class,
```

Step 5 – Now that everything is setup, let's see how we can use various HTML commands using Laravel tags. The first thing to do is to generate a new form by typing the command line

```
{{ Form :: open(array('url' => 'foo/bar')) }}  
//  
{{ Form :: close() }}
```

Next, we give a label to a field (also called as label element) using the `echo form` command

```
echo Form::label('email', 'E-Mail Address');
```

We can create as many such fields and labels as we want. Examples of this is name, last Name, mobile no, etc.- basically all the user information that you wish to capture.

You must have seen while filling up a form that there is grey text in the fields mostly the email id field that says *example@example.com*. You can create this default text by using the following code:

```
echo Form :: text('email', 'example@gmail.com');
```

Sometimes, you need an open-ended box for the user to write like a feedback or suggestion box. These are called as text input boxes and are created by using the following code:

```
echo Form :: text('username');
```